

Ordering Test Methods by Name

JUnit allows **controlling** the **order** in which **test methods run**. In this lesson, the goal is to **order execution alphabetically by method name**.

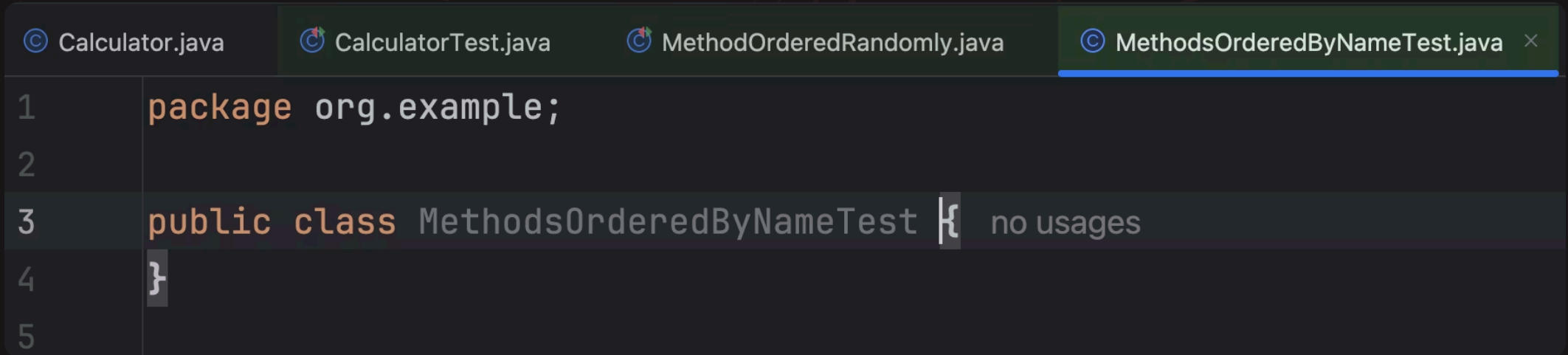
RP

by Rodrigo Martins Pagliares

Last update: 11/27/2025



Creating a new test class



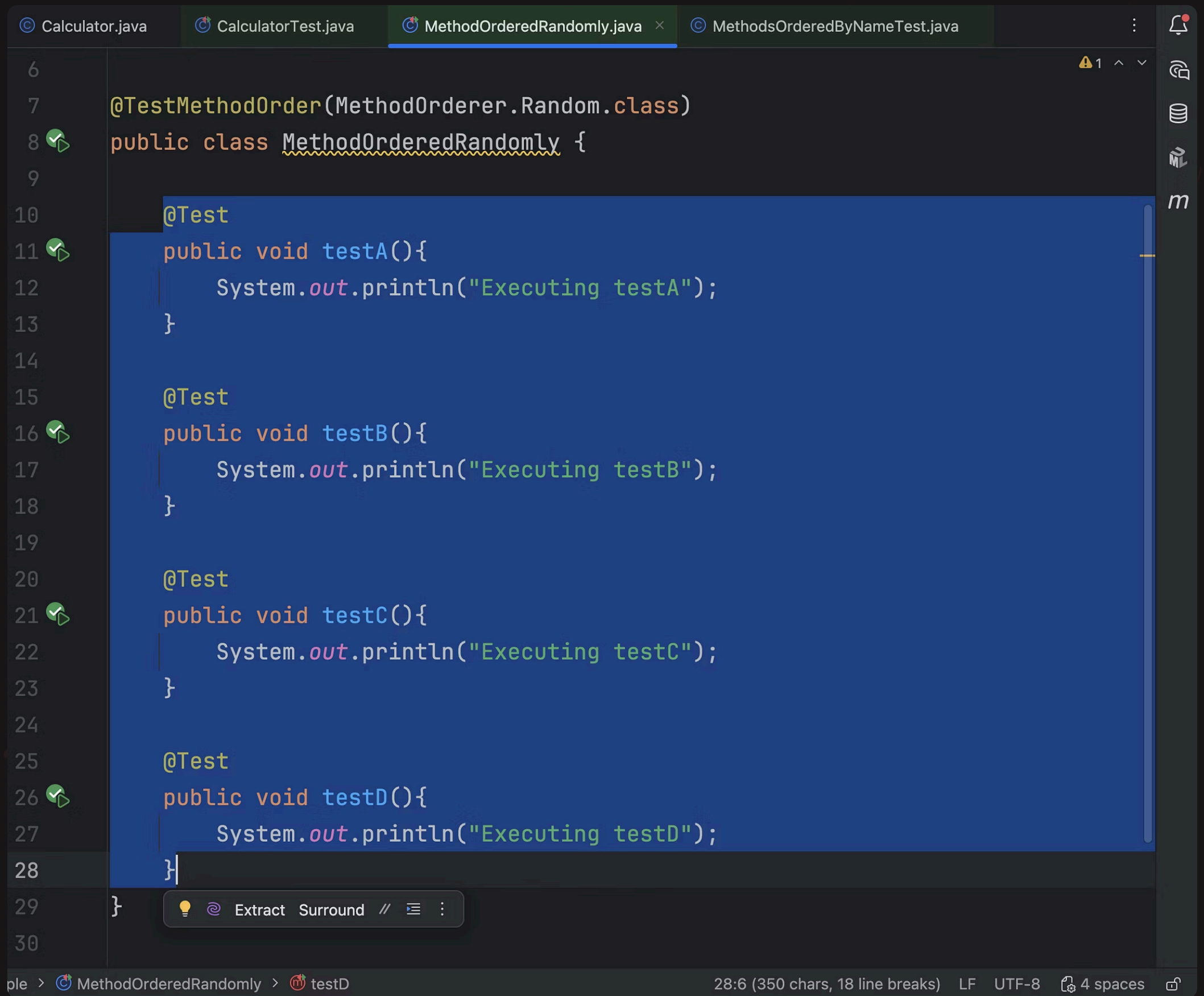
The screenshot shows an IDE with four tabs at the top: Calculator.java, CalculatorTest.java, MethodOrderedRandomly.java, and MethodsOrderedByNameTest.java. The MethodsOrderedByNameTest.java tab is active and highlighted with a blue underline. The code editor displays the following Java code:

```
1 package org.example;  
2  
3 public class MethodsOrderedByNameTest {  
4 }  
5
```

On the right side of the editor, there is a tooltip that reads "no usages".

Creating a new test class (contd.)

Copy the **content** of the **class** created in the **previous lesson**.



The screenshot shows an IDE with four tabs: Calculator.java, CalculatorTest.java, MethodOrderedRandomly.java (active), and MethodsOrderedByNameTest.java. The active tab contains the following Java code:

```
6
7 @TestMethodOrder(MethodOrderer.Random.class)
8 public class MethodOrderedRandomly {
9
10     @Test
11     public void testA(){
12         System.out.println("Executing testA");
13     }
14
15     @Test
16     public void testB(){
17         System.out.println("Executing testB");
18     }
19
20     @Test
21     public void testC(){
22         System.out.println("Executing testC");
23     }
24
25     @Test
26     public void testD(){
27         System.out.println("Executing testD");
28     }
29 }
30
```

The code is highlighted in blue. The IDE interface includes a sidebar on the left with line numbers and a right sidebar with icons for search, run, and other tools. The bottom status bar shows the file path: Calculator > MethodOrderedRandomly > testD, and the file size: 28:6 (350 chars, 18 line breaks). The encoding is UTF-8 and the line ending is LF.

Creating a new test class (contd.)

Past the content in the class being created in this lesson.

```
Calculator.java CalculatorTest.java MethodOrderedRandomly.java MethodsOrderedByNameTest.java x
3 import org.junit.jupiter.api.Test;
4
5 public class MethodsOrderedByNameTest {
6     @Test
7     public void testA(){
8         System.out.println("Executing testA");
9     }
10
11     @Test
12     public void testB(){
13         System.out.println("Executing testB");
14     }
15
16     @Test
17     public void testC(){
18         System.out.println("Executing testC");
19     }
20
21     @Test
22     public void testD(){
23         System.out.println("Executing testD");
24     }
25 }
26
```



Configuring @TestMethodOrder

Annotation Setup

To sort execution by **method name**,
annotate the **class**:

```
@TestMethodOrder(MethodOrderer.M  
ethodName.class)
```

This annotation instructs JUnit to reorder
test execution alphabetically.

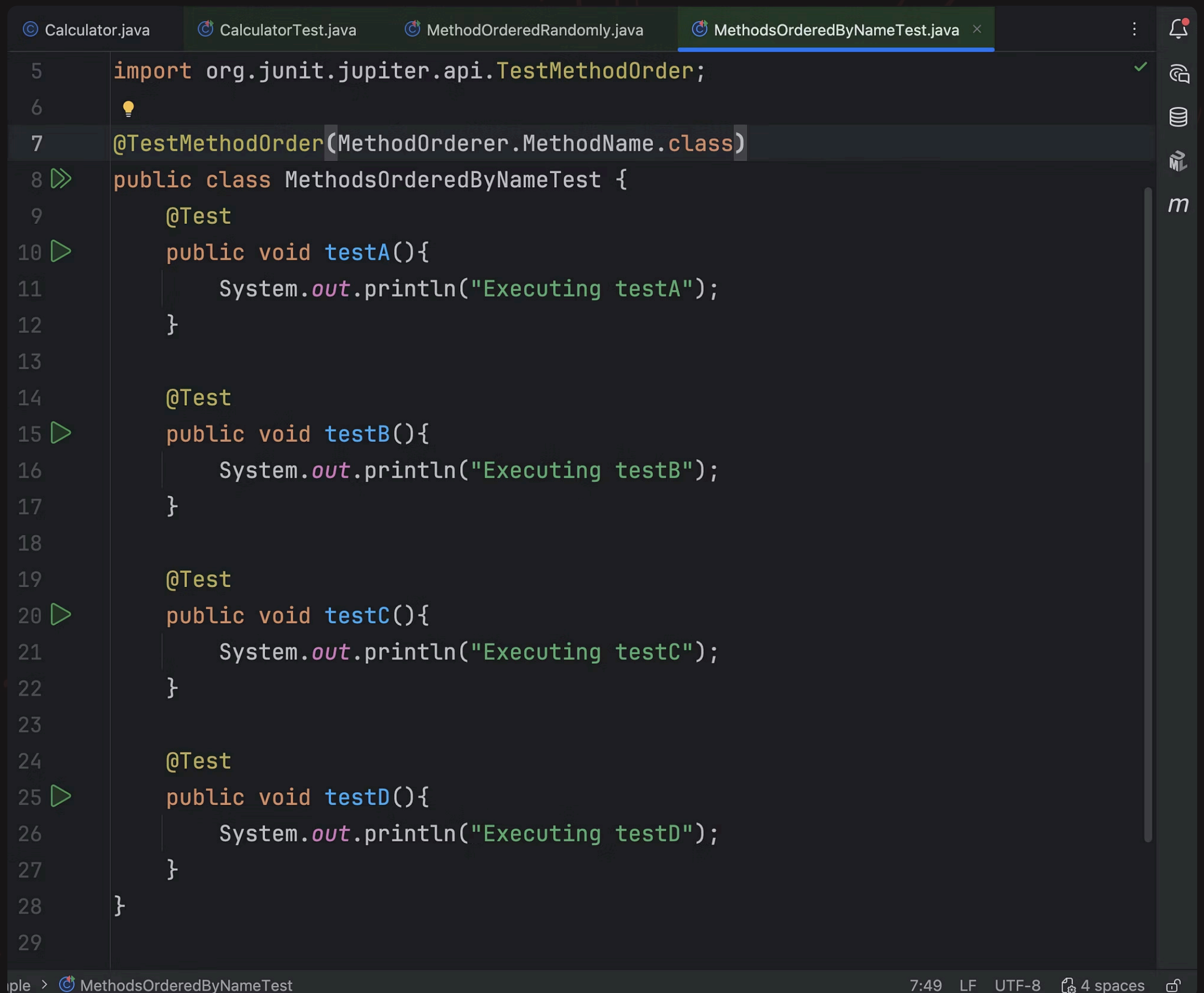
Automatic Sorting

Method names do not need to **appear**
alphabetically in the **file**.

JUnit will still **sort** them **automatically**
during execution.

Configuring @TestMethodOrder

```
@TestMethodOrder(MethodOrderer.MethodName.class)
```



The screenshot shows an IDE with four tabs: Calculator.java, CalculatorTest.java, MethodOrderedRandomly.java, and MethodsOrderedByNameTest.java. The active tab is MethodsOrderedByNameTest.java. The code in the editor is as follows:

```
5 import org.junit.jupiter.api.TestMethodOrder;
6
7 @TestMethodOrder(MethodOrderer.MethodName.class)
8 public class MethodsOrderedByNameTest {
9     @Test
10    public void testA(){
11        System.out.println("Executing testA");
12    }
13
14    @Test
15    public void testB(){
16        System.out.println("Executing testB");
17    }
18
19    @Test
20    public void testC(){
21        System.out.println("Executing testC");
22    }
23
24    @Test
25    public void testD(){
26        System.out.println("Executing testD");
27    }
28 }
29
```

The IDE interface includes a sidebar on the right with icons for a notification bell, a search icon, a database icon, a code icon, and a file explorer icon. The bottom status bar shows the file path 'ple > MethodsOrderedByNameTest', the time '7:49', the line format 'LF', the encoding 'UTF-8', and the indentation '4 spaces'.

Intentionally Scrambled Method Order

Test methods are placed in a **non-alphabetical order** in the class:

- e.g., $B \rightarrow A \rightarrow D \rightarrow C$

This allows **demonstrating** that **JUnit**, not **source order**, controls execution.

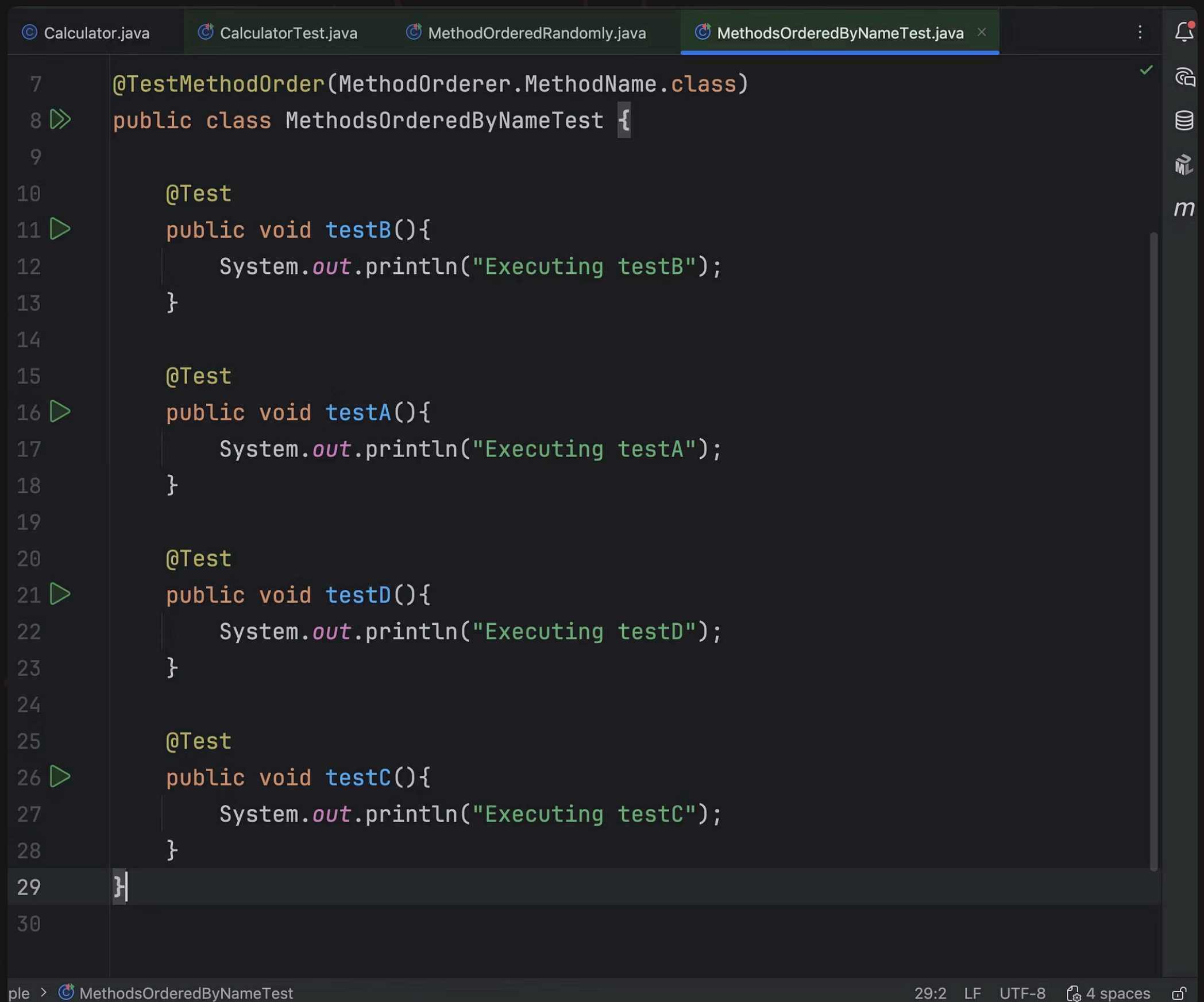
Only the **printed output inside each test method** reveals the order.



Creating a new test class (contd.)

Before **running** the **tests**, **change** the **order** of the **methods** for **demonstration purposes**.

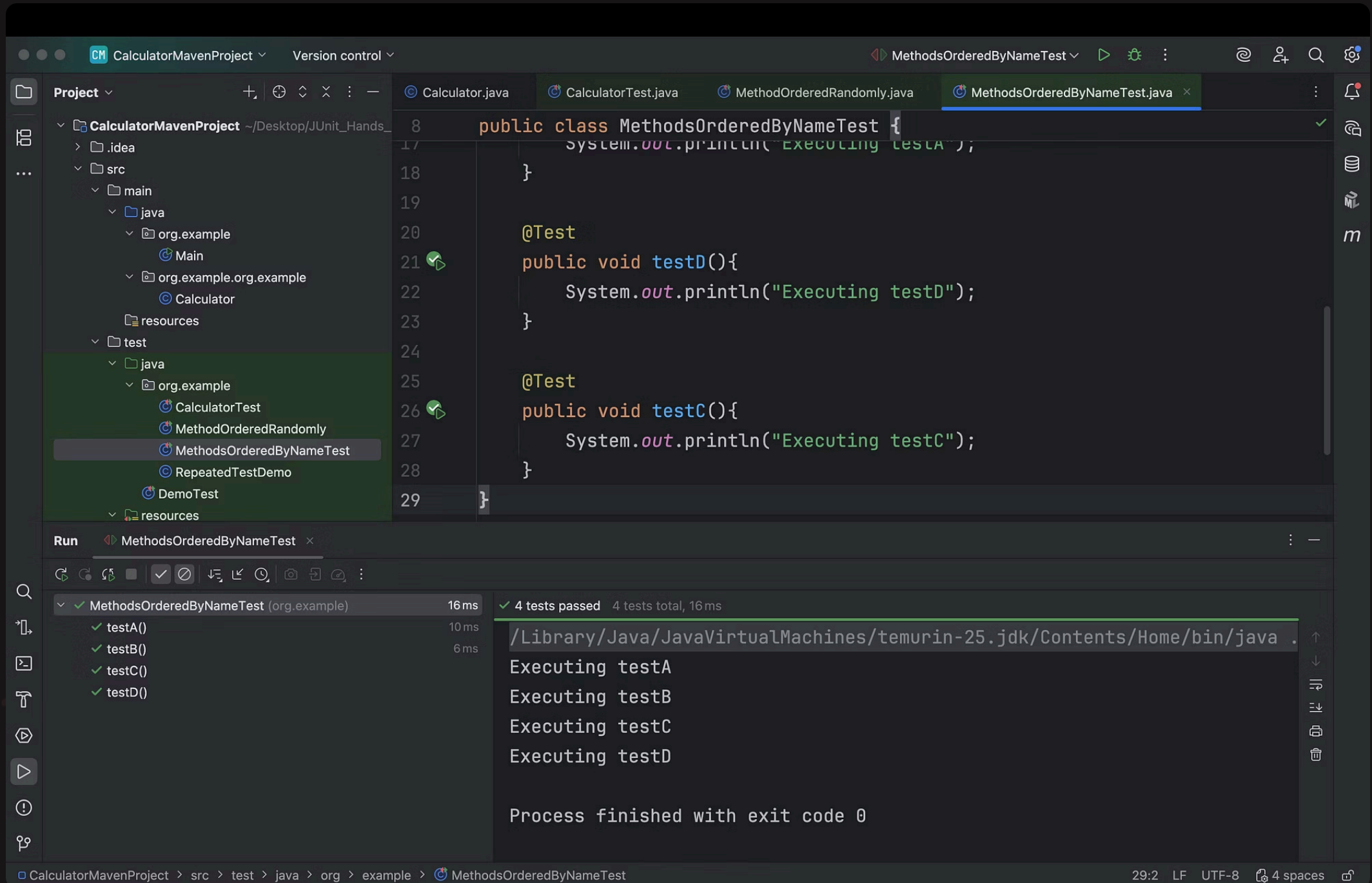
B → A → D → C



```
7  @TestMethodOrder(MethodOrderer.MethodName.class)
8  >> public class MethodsOrderedByNameTest {
9
10     @Test
11     > public void testB(){
12         System.out.println("Executing testB");
13     }
14
15     @Test
16     > public void testA(){
17         System.out.println("Executing testA");
18     }
19
20     @Test
21     > public void testD(){
22         System.out.println("Executing testD");
23     }
24
25     @Test
26     > public void testC(){
27         System.out.println("Executing testC");
28     }
29 }
30
```

ple > © MethodsOrderedByNameTest 29:2 LF UTF-8 4 spaces

Run the test methods



Notice the **output** ordered by **method name**, different from the **order** in the **test code**.

Running the Tests

After running the class, the test report shows:

01

testA

02

testB

03

testC

04

testD

This confirms **alphabetical ordering** regardless of **code arrangement**.

Useful when teams want consistent, predictable test execution order.



Conclusion



JUnit can sort test methods by name using:

```
@TestMethodOrder(MethodOrderer.MethodName.class)
```

Source code order becomes **irrelevant** —
execution follows **alphabetical order**.

This improves test **readability** and **predictability**
when **naming conventions** are **consistent**.



Open Floor Discussion

Question 1

Should alphabetical ordering be used in large test suites?

Question 2

Does enforcing order create hidden dependencies between tests?

Question 3

How does method naming influence the clarity of test execution logs?